From Truth Table to Quantum Circuit: K-Map-Guided Toffoli Synthesis

Arham Akhtar
Lightrider
Bangalore, India
aakhtar@lightriderinc.com

***Abstract*—Karnaugh-map grouping is unsound when applied unchanged to reversible circuits: a classical AND-OR circuit *sets* its output, but a Toffoli circuit *flips* it, so overlapping groups cancel on their shared cells. We give a greedy algorithm that produces an *exact disjoint* cube cover of each output, prove that the resulting Toffoli circuit is correct, and add three function-preserving optimizations, each with a proof: deriving one output from another, cancelling adjacent X gates, and reordering terms so that more X gates cancel. Every circuit is checked by exhaustive simulation. On eleven functions of two to eight inputs the optimizations take a full adder from 23 to 14 gates and a two-bit ripple adder from 126 to 31. We compare against published RevLib circuits and against tweedledum, run on the same functions, and transpile all circuits to a model of a 133-qubit IBM device. The comparison is unfavorable: tweedledum's ESOP circuits have fewer gates on eight of the eleven functions and need fewer CZ gates on the device on seven, and published in-place circuits for `3_17` and `rd32` are 2.2× and 3.5× smaller. The method's value is a construction whose correctness argument is short enough to check by hand.**

***Index Terms*—reversible logic synthesis, Karnaugh maps, Toffoli gates, disjoint cube cover, quantum circuit synthesis, quantum cost**

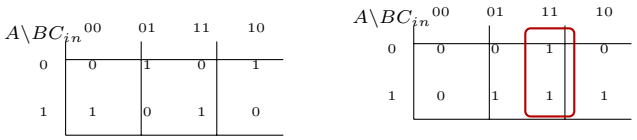
I. Introduction

Grover oracles must flip a target qubit iff a classical predicate $f(x) = 1$, and Shor-style arithmetic chains one-bit adders; both need correct reversible circuits built from truth tables. The textbook tool for minimizing such logic, the Karnaugh map [1], allows overlapping groups. That is sound for a gate that sets its output but not for one that XORs into it: for $\mathrm{Out} = A + B$, the naive circuit `cx(B,Out); cx(A,Out)` returns 0 on A=B=1.

Our contributions are: (1) a greedy algorithm producing an exact disjoint cube cover, with proofs that it and the circuit built from it are correct; (2) output derivation, X-cancellation, and term reordering, each proved function-preserving, with derivation chosen by the gate count left after cancellation; (3) an evaluation on eleven functions against published RevLib circuits, against tweedledum run on the same functions, and on an IBM device model, which reports where our circuits are larger. The code that produces every number here is at https://github.com/ArhamAkhtarAlam/toffoli-synthesis-final-paper.

II. Related Work

Reversible synthesis from truth tables [2] includes transformation-based synthesis [3], ESOP-based synthesis (e.g. EXORCISM-4 [4]), BDD-based synthesis [5], and exact SAT-based synthesis [6]. A disjoint cube cover is a restricted ESOP: it gives up the smaller circuits a general ESOP search finds in exchange for a short correctness proof. RevLib [7] collects benchmarks and published circuits; RevKit [8] and tweedledum [9] implement these synthesis families, and Qiskit [10] exposes truth-table oracles. Downstream, compilers such as t|ket⟩ [11] and Qiskit's transpiler map circuits to a device's gates and connectivity, and exact search finds optimal Clifford+T circuits for small gates such as the Toffoli [12]. Our circuits are an input to that stage.



Sum: no two 1-cells adjacent (parity).

Carry: group BC_{in}; the other two 1-cells must be single terms.

Fig. 1. K-maps for the full adder. No two of Sum's 1-cells are adjacent; Carry's disjoint cover is the boxed pair and two single cells.

III. Disjoint Covers and Circuits

A *pattern* $p \in \{0, 1, -\}^n$ denotes the cube $[\![p]\!] = \{x : x_k = p_k \text{ whenever } p_k \neq -\}$. Circuits act on n input, m output, and a ancilla wires (outputs and ancillas start at 0) with gates X and C^kX (flip the target iff all k controls are 1): the positive-control MCT library of RevLib [7]. A term p becomes one C^kX on the wires where p is specified, wrapped in X gates on those where $p_k = 0$, so it flips its target iff $x \in [\![p]\!]$. The quantum cost of a C^kX is $2^{k+1} - 3$ (1 for X and CNOT) [13], [7]; on a fault-tolerant target a 2-control Toffoli costs 7 T and 6 CNOT gates [14], and larger C^kX decompose into it.

A K-map lays out a truth table so that adjacent cells differ in one bit; a group of 2^d adjacent 1-cells is a cube with d dashes. Classical grouping lets groups overlap, but a Toffoli circuit flips a cell once per group containing it, so a cell in two groups ends at 0 (cell 11 of $A + B$). Overlaps are harmless only when every cell is covered an odd number of times, as for Carry's three pairs, which meet only at 111: that is an ESOP (Section II). Algorithm 1 instead grows each group only inside the cells not yet covered (Fig. 1).

Theorem 1 (Exact, disjoint cover)**.** *The cubes returned by Algorithm 1 are pairwise disjoint and their union is S.*

Algorithm 1 DISJOINTCOVER(S)

$R \leftarrow S$; $T \leftarrow \langle\rangle$
while $R \neq \emptyset$ **do**
  $p \leftarrow \min(R)$
  for $i \leftarrow 1$ **to** n **do** ▷ GROWCUBE
    if $[\![p$ with $p_i{=}-]\!] \subseteq R$ **then** $p_i \leftarrow -$
  end for
  append p to T; $R \leftarrow R \setminus [\![p]\!]$
end while
return T

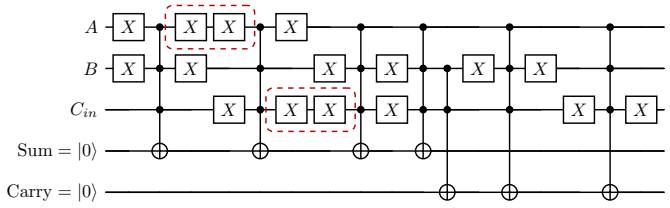


Fig. 2. Full adder as emitted: Sum's four terms, then Carry's three (23 gates). In each dashed box, one term ends and the next begins with an X on the same wire, and no gate between them uses that wire as a control; since $XX = I$, both are deleted (Lemma 2), leaving 19 gates.

Proof. Each cube is grown only while it stays inside the current remainder R, which excludes all earlier cubes, so cubes are disjoint and lie in S. Each iteration removes at least its seed, so the loop ends, with $R = \emptyset$, after at most $|S|$ iterations. □

Theorem 2 (Correct synthesis)**.** *Emitting every term of* DISJOINTCOVER(f_j) *onto output wire j, for each j, maps $(x, 0^m)$ to $(x, f(x))$ for every x.*

Proof. A term flips wire j iff x is in its cube. The cubes are disjoint and cover the on-set (Theorem 1), so wire j flips exactly once iff $f_j(x) = 1$. No gate targets an input wire. □

Fig. 2 shows the full adder: 4 singleton terms for Sum (parity admits no larger cube) and 3 for Carry, 23 gates in all.

IV. OPTIMIZATIONS

Derivation. A later output f_k may be built as a CNOT from an earlier output wire j followed by the cover of $r = f_j \oplus f_k$.

Lemma 1 (Derivation)**.** *If wire j holds $f_j(x)$, a CNOT $j \to k$ followed by the terms of* DISJOINTCOVER(r) *on wire k leaves $f_k(x)$ on k and changes no other wire.*

Proof. Wire k ends at $f_j(x) \oplus r(x) = f_k(x)$ by Theorem 2 applied to r. □

On a two-bit ripple adder (S_0, C_0, S_1, Cout) derivation finds $S_1 \oplus C_0 = A_1 \oplus B_1$, the identity "sum = inputs XOR carry," and cuts 126 gates to 64 (Table II). A term common to several outputs can also be computed once into an ancilla, copied, and uncomputed; for two outputs this costs $2c + 2$ gates against $2c$, so it almost never pays and no circuit here uses it.

X-cancellation. Adjacent terms often leave $X_w \cdots X_w$ on an input wire w, as in the two boxed pairs of Fig. 2.

Lemma 2 (X-cancellation)**.** *If no gate between two X_w uses w as a control, deleting both leaves the circuit's action unchanged.*

Proof. X_w commutes with gates not involving w and with gates targeting w (both flip w; their condition does not depend on w), so the two X_w become adjacent and $X_w X_w = I$. □

Since X gates sit only on inputs, the cancelled length is the number of $C^k X$ gates plus, per wire, the parity of the X gates between consecutive control uses, computed in one pass. Chosen by gate count *before* cancellation, derivation can hurt: the full adder goes from 19 (cancellation alone) to 22. We therefore score each derivation candidate by the cancelled length of the circuit so far, and fall back to the plain cover whenever it is shorter, so the pipeline is never worse than cover plus cancellation. On the ripple adder this derives Cout instead of S_0 from C_0: one gate more before cancellation, but 47 gates after it instead of 58 (Table II).

Term ordering. Call each term's gates, and each derivation CNOT, a *block*.

Lemma 3 (Reordering)**.** *Any block order in which each derivation CNOT from output i follows every block writing wire i computes the same function.*

Proof. Each block XORs a function of wires it does not write into the wires it writes, so two blocks commute unless one writes a wire the other reads; only a derivation CNOT reads a non-input wire. Any order meeting the constraint is reached from the emitted one by swapping adjacent commuting blocks. □

Our heuristic builds an order greedily (append the admissible block that gives the shortest cancelled prefix), then moves single blocks while the cancelled length drops. For reference, dynamic programming over (placed blocks, per-wire X parity) gives the exact optimum in $O(2^B 2^n B)$ for B blocks; for the full adder's cover it matches exhaustive search over all 7! orders (15 gates).

V. EVALUATION

Setup. The method is implemented in Python. The functions are the six built-in presets (AND, OR, XOR, NAND, half and full adder), a two-bit ripple adder with all four internal signals as outputs, BCD-to-Excess-3, a 3×3-bit multiplier, a 4-bit equality comparator (the shape of a Grover oracle predicate), and RevLib's `3_17` [7]. Every circuit, before and after each pass, is checked against its truth table on all 2^n inputs by classical replay, which is exact because every gate maps basis states to basis states. Algorithm 1 was also checked on 2000 random on-sets ($n \leq 6$) with no overlap or gap. Over 2000 random truth tables ($2 \leq n \leq 4$, $1 \leq m \leq 3$), cancellation shortens 47.2% of cover circuits (9.7% on average); without the post-cancellation choice the pipeline is worse than cover plus cancellation in 2.4% of them, and with it in none; ordering

TABLE I

GATE COUNT AFTER X-CANCELLATION (COVER, FULL PIPELINE, WITH TERM ORDERING, EXACT BEST ORDER; – WHERE $B > 16$), AND GATES, QUANTUM COST, AND CZ COUNT ON THE IBM HERON MODEL FOR OUR FINAL CIRCUIT AND TWEEDLEDUM'S PKRM AND PPRM CIRCUITS ON THE SAME $n + m$ LINES.

			This work: gates					tweedledum gates		Q-cost			CZ on device		
Function	n	m	Raw	Cover	Full	Ord.	Opt.	PKRM	PPRM	Ours	PKRM	PPRM	Ours	PKRM	PPRM
AND	2	1	1	1	1	1	1	1	1	5	5	5	7	7	7
OR	2	1	7	4	4	4	4	4	3	8	8	7	7	10	8
XOR	2	1	6	6	6	6	6	2	2	14	2	2	17	2	2
NAND	2	1	11	6	6	6	6	6	2	10	10	6	7	8	7
Half adder	2	2	7	7	6	6	6	3	3	14	7	7	19	14	14
Full adder (rd32)	3	2	26	19	19	14	14	10	6	74	22	18	133	34	36
2-bit ripple adder	5	4	332	86	47	31	–	34	18	411	142	118	514	243	231
BCD to Excess-3	4	4	122	36	28	22	22	45	32	190	257	292	279	388	493
3×3 multiplier	6	6	615	123	97	60	–	80	27	1920	936	919	2058	1235	1039
4-bit equality	8	1	144	76	76	52	48	24	81	8180	472	1009	7534	752	1768
3_17	3	3	48	22	15	13	13	15	14	37	31	30	65	44	49

TABLE II

TWO-BIT RIPPLE ADDER: GATES PER OUTPUT BEFORE X-CANCELLATION, WITH DERIVATION CHOSEN BEFORE OR AFTER CANCELLATION (BASE IN PARENTHESES).

Output	Direct	Before-X rule	After-X rule
S_0	16	15 (C_0)	16
C_0	7	7	7
S_1	68	7 (C_0)	7 (C_0)
Cout	35	35	35 (C_0)
Total	126	64	65
After X-cancellation	86	58	47

TABLE III

AGAINST PUBLISHED CIRCUITS (QUOTED FROM REVLIB) AND TWEEDLEDUM.

Function	Method	Lines	Gates	Q-cost
3_17	MMD, MCT [3]	3	6	14
	Exact, elementary gates [6]	3	10	10
	tweedledum transformation	3	7	15
	tweedledum PPRM	6	14	30
	This work	6	13	37
rd32	Best MCT on RevLib [7]	4	4	12
	tweedledum PPRM	5	6	18
	This work	5	14	74

removes a further 9.4% and reaches the exact optimum in 94.9% of the 1904 circuits where that optimum was computed. Derivation is used in 48.7% of the 1326 tables with two or more outputs, cutting gates before cancellation by 22.5% when used; sharing is chosen in 0.4%. The checker accepted every circuit and rejected each one against its table with one output bit flipped.

Our pipeline. Table I (Raw: one term per on-set row, before cancellation) shows the passes compounding: the multiplier goes from 615 to 60 gates. On the comparator no cube larger than a minterm exists, so the whole gain ($144 \to 52$) comes from cancellation and ordering; ordering stops at 52 there against an exact 48.

tweedledum. We ran tweedledum 1.1.1 [9]'s PKRM and PPRM synthesizers, which build each output as an ESOP on its own line, rewrote negative controls as X-sandwiches, applied our cancellation pass, and verified each circuit. Its better circuit has fewer gates than ours on eight functions; ours has fewer only on BCD-to-Excess-3 (22 vs. 32) and 3_17 (13 vs. 14), and lower quantum cost only on BCD-to-Excess-3. The comparator is the worst case (Q-cost 8180 vs. 472): each of its terms is an 8-control minterm, while an ESOP shares structure across terms.

Published circuits (Table III). For the bijection 3_17, RevLib lists a Miller–Maslov–Dueck circuit [3] with 6 gates (Q-cost 14) on 3 lines and an exact elementary-gate circuit with Q-cost 10 [6]; tweedledum's transformation-based synthesis gives 7 gates (Q-cost 15). For rd32, the full adder, the best listed MCT circuit has 4 gates (Q-cost 12) on 4 lines. Ours are 2.2× and 3.5× larger in gates and 2.6× and 6.2× in quantum cost, partly because they are out-of-place ($n + m$ lines, inputs preserved) while the published ones are in-place or keep garbage. These circuits are quoted from RevLib, not re-run.

On a device. We transpiled every circuit with Qiskit 2.6 to FakeTorino, Qiskit's offline model of the 133-qubit IBM Heron processor ibm_torino (native CZ, $\sqrt{X}$, R_z; its coupling map), at optimization level 3 with a fixed seed, and checked each transpiled circuit, placement and routing included, by statevector simulation on all inputs. The transpiler must be told that qubits do not start in $|0\rangle$: by default it borrows untouched input wires as clean scratch qubits, and the transpiled ripple adder is then wrong on some inputs. Our circuits need fewer CZ gates than tweedledum's better one only on OR and BCD-to-Excess-3 (279 vs. 388), tie on AND and NAND, and need more on the other seven. Which circuit needs fewer CZ gates matches which has lower quantum cost in 18 of the 22 comparisons, and which has fewer MCT gates in only 13: on 3_17 ours has fewer gates but 65 CZ gates against 44. Quantum cost is the better proxy. Circuit depth follows the CZ count (e.g. 1409 against 701 for the ripple adder, 760 against 1104 for BCD-to-Excess-3).

VI. Limitations and Conclusion

Requiring Karnaugh groups to be disjoint makes them sound for XOR-accumulating gates, with a proof short enough to check by hand, and derivation, cancellation, and reordering keep that guarantee (Lemmas 1–3). The method is not competitive on gate count, quantum cost, or device CZ count with tweedledum or with published in-place circuits. Its choices are greedy: the cover, the derivation base (one at a time, before ordering, for gate count rather than quantum cost), and the ordering, which misses the optimum in 5.1% of random trials. Closing the gap needs non-disjoint ESOP terms, cost-aware cube growth, and an in-place variant. We did not run RevKit, the device results use one transpiler, one device model, and one seed, and all functions have $n \leq 8$.

References

[1] M. Karnaugh, "The map method for synthesis of combinational logic circuits," *Trans. AIEE, Part I*, vol. 72, no. 5, pp. 593–599, 1953.

[2] M. Saeedi and I. L. Markov, "Synthesis and optimization of reversible circuits—a survey," *ACM Comput. Surv.*, vol. 45, no. 2, art. 21, 2013.

[3] D. M. Miller, D. Maslov, and G. W. Dueck, "A transformation based algorithm for reversible logic synthesis," in *Proc. 40th DAC*, pp. 318–323, 2003.

[4] A. Mishchenko and M. Perkowski, "Fast heuristic minimization of exclusive-sums-of-products," in *Proc. 5th Int'l Reed-Muller Workshop (RM)*, pp. 242–250, 2001.

[5] R. Wille and R. Drechsler, "BDD-based synthesis of reversible logic for large functions," in *Proc. 46th DAC*, pp. 270–275, 2009.

[6] D. Große, R. Wille, G. W. Dueck, and R. Drechsler, "Exact synthesis of elementary quantum gate circuits for reversible functions with don't cares," in *Proc. 38th Int'l Symp. on Multiple-Valued Logic (ISMVL)*, pp. 214–219, 2008.

[7] R. Wille, D. Große, L. Teuber, G. W. Dueck, and R. Drechsler, "RevLib: An online resource for reversible functions and reversible circuits," in *Proc. 38th Int'l Symp. on Multiple-Valued Logic (ISMVL)*, pp. 220–225, 2008.

[8] M. Soeken, S. Frehse, R. Wille, and R. Drechsler, "RevKit: An open source toolkit for the design of reversible circuits," in *Reversible Computation 2011*, LNCS vol. 7165, Springer, pp. 64–76, 2012.

[9] B. Schmitt and G. De Micheli, "tweedledum: A compiler companion for quantum computing," in *Proc. DATE*, pp. 7–12, 2022.

[10] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[11] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, "t|ket⟩: A retargetable compiler for NISQ devices," *Quantum Sci. Technol.*, vol. 6, no. 1, art. 014003, 2021.

[12] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 6, pp. 818–830, 2013.

[13] A. Barenco *et al.*, "Elementary gates for quantum computation," *Phys. Rev. A*, vol. 52, no. 5, pp. 3457–3467, 1995.

[14] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge Univ. Press, 2000.